

**LINUX  
SCENARIO  
BASED Q&A  
DAY 5**

## Q1. ps Shows a Process, But You Can't Identify the Script or Command Behind It

🧩 Scenario: A process is visible via `ps` by its PID, but it's unclear what command or script it's running.

🧠 Possible Root Cause:

- It could be a forked shell, background job, or a binary without a clear path or arguments shown.

🔧 Solution:

1. Get more detail on the process:

```
ps -p <PID> -o cmd
lsof -p <PID>
readlink /proc/<PID>/exe
```

2. Use tools like `strace` for live system call tracing:

```
strace -p <PID>
```

✅ Tip:

Combine `lsof`, `readlink`, and `strace` to get a full picture of what a mysterious process is doing and where it originated from.

## Q2. Script Runs Indefinitely Without Completing

🧩 Scenario: A shell script keeps running with no output and doesn't exit, making it hard to tell what's going wrong.

🧠 Possible Root Cause:

- Infinite loop or
- A `read` or blocking command waiting for user input

🔧 Solution:

1. Attach `strace` to the process to see what it's doing:

```
strace -p <PID>
```

2. Add debugging lines to the script:

```
set -x # Enable debug mode
echo "Checkpoint: Reached line X"
```

✅ Tip:

Always include timeout logic or watchdog checks in critical automation scripts to prevent them from hanging silently.

## Q3. Only One Background Process Runs from a Script

🧩 Scenario: You have a script meant to start multiple background processes (e.g., 3-4), but only the first one runs as expected.

🧠 Possible Root Cause:

- Incorrect use of `&` or process chaining, later commands may be blocked or not properly backgrounded.

🔧 Solution:

Use the correct syntax to launch each command in the background: `command1 &` and `command2 &`

Add `wait` if you need the script to wait for all background processes to complete before exiting.



Tip:

Use trap to handle cleanup gracefully if the script is interrupted:

```
trap 'kill 0' SIGINT SIGTERM
```

#### Q4. Scheduled Process or Cron Job Gets Killed Randomly

✿ Scenario: A background task or cron job starts but gets terminated unexpectedly before completion.

🧠 Possible Root Cause:

- Out-of-Memory (OOM) killer, insufficient memory, or restrictive ulimit settings.

🔧 Solution:

1. Check system logs for signs of forced termination:

```
dmesg | grep -i kill
```

2. Adjust memory limits if necessary:

```
ulimit -v unlimited
```



Tip:

For critical workloads, use cgroups to set fine-grained memory and CPU limits and avoid unexpected terminations.

#### Q5. Service Doesn't Start Automatically at Boot

✿ Scenario: The system boots up fine, and the service starts manually without issues, but it fails to start on its own during boot.

🧠 Possible Root Cause:

- The service is not enabled to start at boot, or its systemd dependencies (e.g., After=) are misconfigured.

🔧 Solution:

1. Enable the service to auto-start:

```
sudo systemctl enable myservice
```

2. Check and set proper dependency ordering in the unit file (if needed):

```
After=network.target
```



Tip:

Use systemd-analyze blame to identify delays or blockers in the boot sequence that might be affecting your service.

#### Q6. Process Crashes Exactly One Minute After Starting

✿ Scenario: A service or process launches successfully, runs for about 60 seconds, then crashes or stops unexpectedly.

🧠 Possible Root Cause:

- WatchdogSec timeout in systemd
- Application-level memory leak
- Failure during automatic config reload

 Solution:

1. Check detailed logs for clues:

```
journalctl -xe
```

2. Review the unit file for watchdog settings:

```
WatchdogSec=60
```

3. Run the application manually (outside systemd) to verify if the issue is internal to the app.

 Tip:

Avoid setting watchdog timers unless you have a proper watchdog mechanism in place. Otherwise, systemd may terminate the service due to missing keep-alive signals.

## Q7. How to Auto-Restart a Script on Crash

 Scenario: You want a script to automatically restart if it crashes or exits unexpectedly.

 Possible Root Cause:

- By default, shell scripts exit silently on failure and are not managed by any supervisor.

 Solution:

1. Wrap the script in a systemd service for better reliability and control:

```
[Unit]Description=My Resilient Script
[Service]ExecStart=/path/to/script.sh
Restart=always
RestartSec=5[Install]WantedBy=multi-user.target
```

2. Then enable it to start at boot:

```
sudo systemctl enable myscript.service
sudo systemctl start myscript.service
```

 Tip:

Using systemd is far more robust than relying on while true loops within the script.

## Q8. Process Spawns Too Many Threads, Slowing Down the System

 Scenario: A single application is creating 1000+ threads, leading to system lag or resource exhaustion.

 Possible Root Cause:

- Likely a thread leak or misconfigured concurrency setting in the application.

 Solution:

1. Check the thread count for the process:

```
ps -eLf | grep appname | wc -l
```

2. Review and reduce the thread/concurrency settings in the app's configuration.

 Tip:

Use `ulimit -u` to set a cap on the number of user-level processes/threads and prevent runaway resource usage.

### Q9. Kernel Panic on Boot: “Unable to Mount Root FS”

🌿 Scenario: On boot, the system immediately halts or reboots with a kernel panic error saying it can't mount the root filesystem.

🧠 Possible Root Cause:

The kernel is unable to locate or mount the root partition, often due to filesystem corruption, missing drivers, or a broken initramfs.

🔧 Solution:

1. Boot into a Live CD or rescue mode.
2. Run a filesystem check and repair:  
`fsck /dev/sdXn`
3. Mount the root partition, chroot into it, and rebuild the initramfs:  
`chroot /mnt`  
`update-initramfs -u`

✅ Tip:

Always keep a rescue ISO or snapshot available for quick recovery from boot-level failures.

### Q10. System Boot Takes Over 5 Minutes

🌿 Scenario: The system takes unusually long to boot, often stalling on specific services.

🧠 Possible Root Cause:

- Services timing out (e.g., network, NFS mounts) or incorrect disk UUIDs in `/etc/fstab`.

🔧 Solution:

1. Analyze boot performance:  
`systemd-analyze blame`
2. Identify and disable problematic services:  
`sudo systemctl disable service-name`

✅ Tip:

Use `journalctl -b` to dig into boot-time logs and spot delays or failures more accurately.